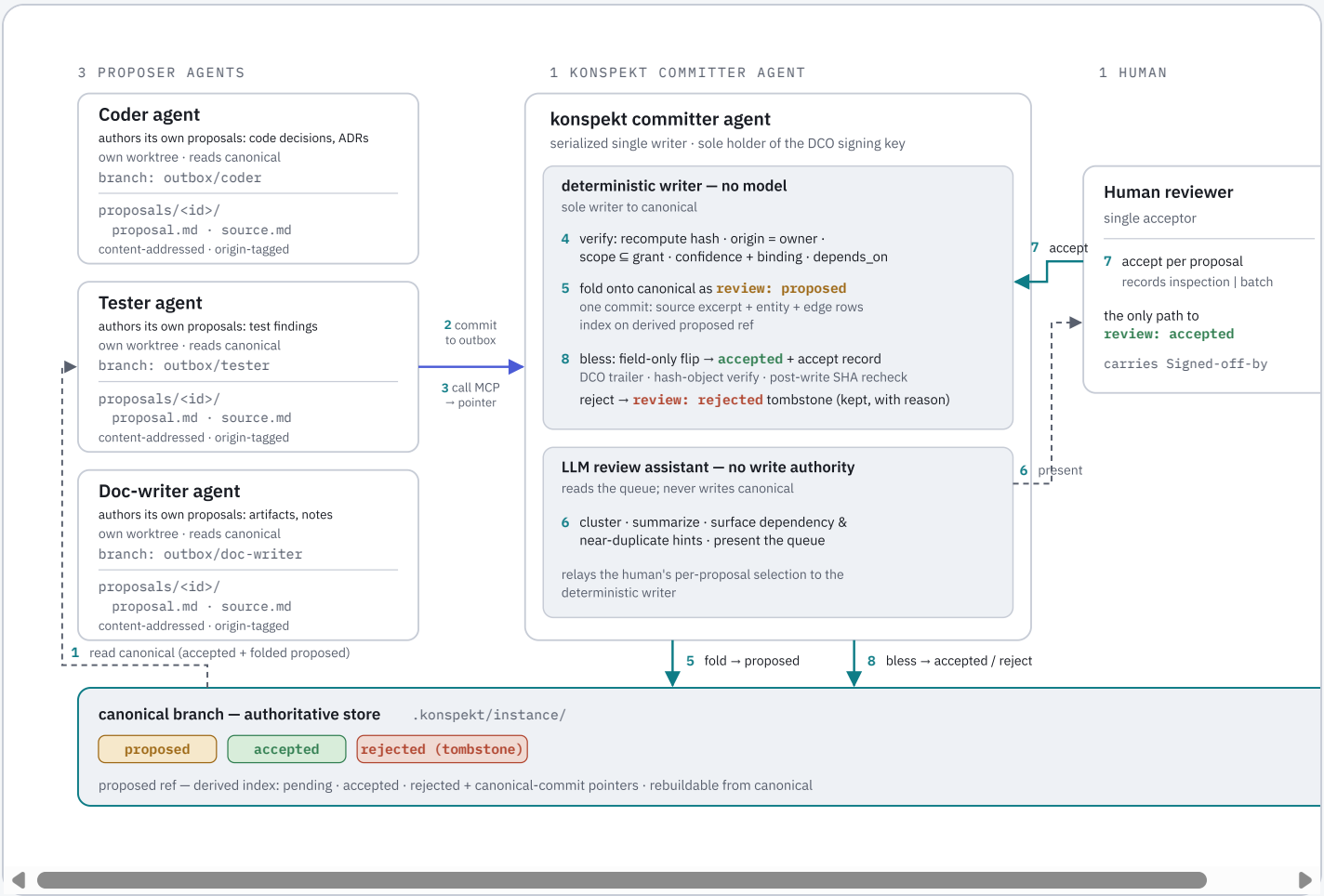
KONSPEKT · TASK-AGENT-FLEET

One reviewer over a fleet of agents

Three proposer agents — a coder, a tester, and a doc writer — propose into one konspekt instance. A single committer agent verifies, folds, and (on one human's acceptance) blesses every proposal. Agents never write canonical; the committer is the only writer.

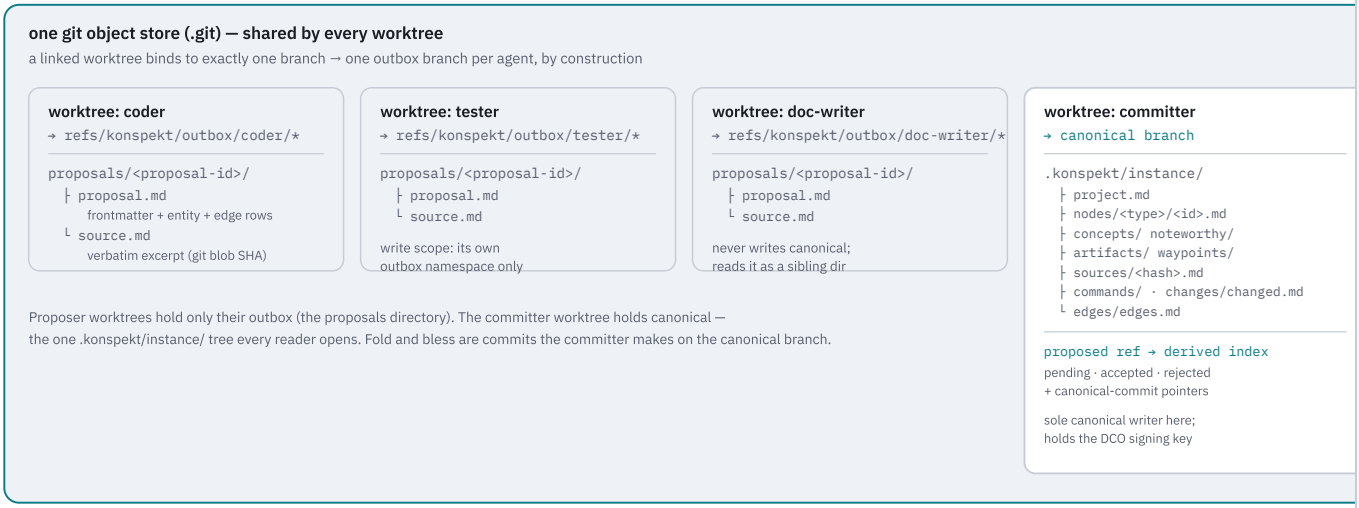
FIGURE 1 — HOW A PROPOSAL MOVES



canonical write (committer only) — MCP signal (pointer) --- read

Sync is one-directional: **outbox → committer → canonical**. The MCP call carries a pointer, not the payload; the committer reads the bytes from the store it already trusts and verifies the hash, so a replayed call resolves to the same proposal and is deduplicated. The **review: accepted** transition is human-only; the review assistant organizes the queue but holds no write authority.

FIGURE 2 — REPOSITORY LAYOUT: N+1 WORKTREES OFF ONE OBJECT STORE



The same shared object store backs all four worktrees, so a proposer reads canonical as a local sibling-directory read while writing only to its own outbox branch. **Canonical is authoritative** for proposal state; the **proposed ref** is a derived index the committer can rebuild by scanning `review` fields and accept records.

THE COMMITTER PROTOCOL, STEP BY STEP

Proposer side

1. **Read canonical.** The agent reads the accepted graph plus every peer proposal folded so far, as a local sibling-directory read.
2. **Commit the payload.** It writes `proposal.md` and `source.md` under `proposals/<id>/` on its own outbox branch. The commit gives the proposal durability before any call.
3. **Call over MCP.** It signals the committer with a pointer (content hash or outbox ref). Commit-then-call is the proposer's choice; an agent with no writable store sends the payload in the call instead.

Committer side

4. **Verify.** Recompute the hash, confirm `origin` equals the namespace owner, confirm scope is within the agent's grant, confirm `confidence` and a resolved `binding` are present, resolve `depends_on`. A failed check sets the proposal aside with a reason.

5 **Fold.** Fold the verified proposal onto canonical as `review: proposed` in one commit — source excerpt, entity file, edge rows — and index it on the proposed ref.

6 **Present.** The review assistant clusters and summarizes the queue and puts it to the human. It holds no write authority.

7 **Accept.** The human names the subset to admit, per proposal, recording whether the decision was by inspection or by batch, carrying `Signed-off-by`.

8 **Bless or reject.** For each admitted proposal, a field-only commit flips `review` to `accepted` and appends the accept record. A declined proposal becomes a `rejected` tombstone, retained with its reason.

INVARIANTS THE LAYOUT HOLDS

- **Proposers never write canonical.** Sync is one-directional: outbox → committer → canonical. The committer is the only writer to canonical and the only holder of the DCO signing key.
- **Least-privilege write scope.** A proposer holds write to its own outbox namespace only (`refs/konspekt/outbox/<agent-id>/*`); canonical and the proposed ref stay committer-only.
- **Canonical is authoritative.** Proposal state lives in each atom's `review` field on canonical; the proposed ref is a derived, rebuildable index, so a crash between a fold commit and a ref update cannot make the store disagree with itself.
- **Content-addressed, idempotent delivery.** The MCP call carries a pointer; the committer fetches the bytes and verifies the hash on read, so a replayed call resolves to the same proposal and is deduplicated.
- **Origin is stamped, not trusted.** The committer records `origin` from the authenticated owner of the outbox namespace, and checks scope against the grant as triage — not as an enforcement boundary.
- **Human-only acceptance.** One acceptor makes every `review: accepted` transition; rejection is a retained tombstone, never a delete.
- **Contained egress.** The fleet runs inside an egress sandbox with the committer host as the single declared egress; konspekt supplies the accountable payload on that egress.

Not tied to one agent framework

This layout is not specific to any one agent runtime. Any multi-agent framework can drive it, as long as each proposer agent is given a local repository, branch, and worktree that are set up correctly and that carry a `.konspekt` instance — the framework's only job is to provision those and point each agent at its own outbox branch.

An agent that has no local worktree (a web or mobile conversation, for example) is not excluded: it reaches the committer over MCP and sends its proposal in the call. The committer, the propose-then-accept flow, and the content-addressed payload are identical in both cases. Only how each agent is provisioned changes.